



Network

University of Insubria

ISO/OSI and TCP/IP

ISO/OSI and TCP/IP represent the reference models for communication between different computers in the network

- separate networking functions into logical smaller pieces
- provide modularity and extensibility

Note that

- ISO/OSI model evolved as a theoretical model
- TCP/IP as a practical model

OSI Layers

Application

It provides the services to the user

Presentation

It is responsible for the formatting of information (e.g., compression and encryption)

Session

It is responsible for establishing, managing, and terminating sessions

Transport

It provides message delivery from process to process

Network

It is responsible for moving the packets from source to destination

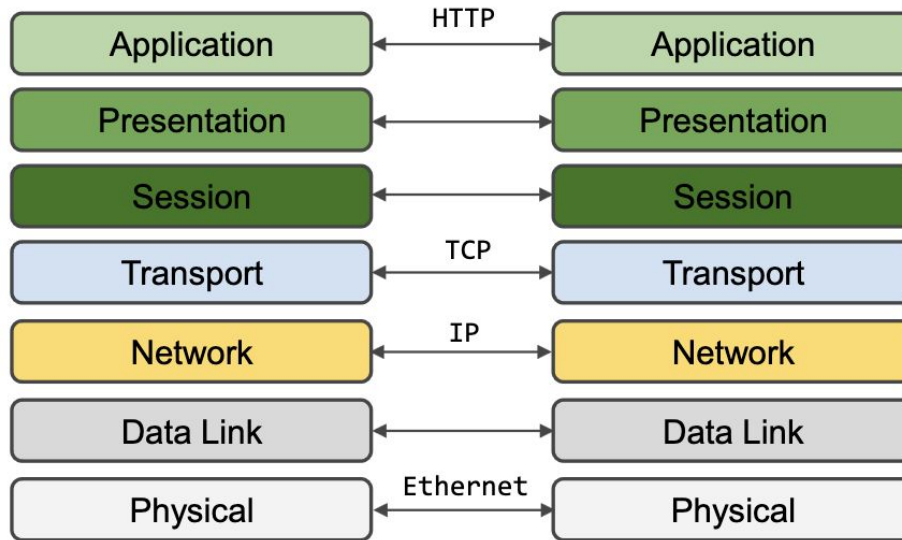
Data Link

It combines bits into a structure of data and provides their error-free transfer

Physical

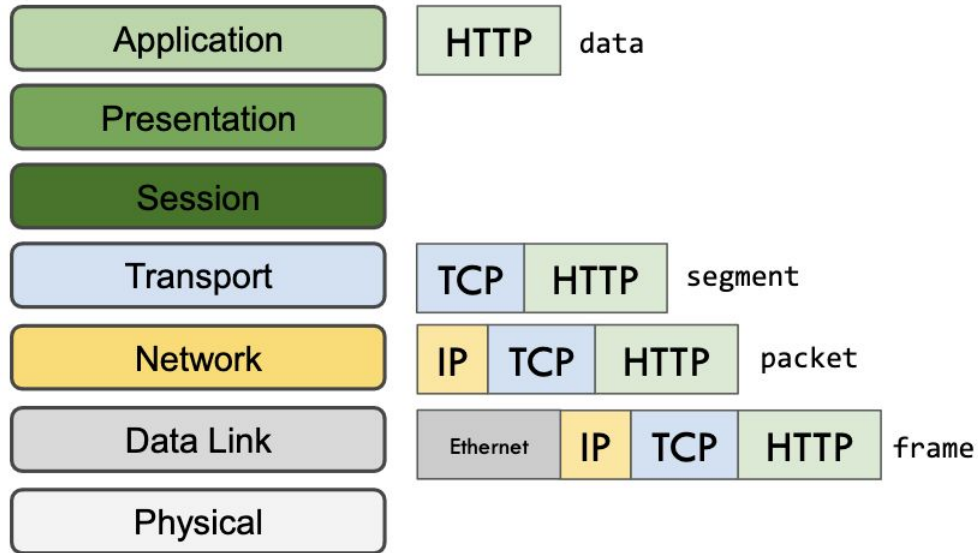
It provides a physical medium through which bits are transmitted

OSI Layers



- The initial data transfer begins at the application layer of the transmitter
- Each layer can communicate just with the layers directly above and below it
- A protocol is a standard procedure and format that two data communication devices must understand

OSI Layers



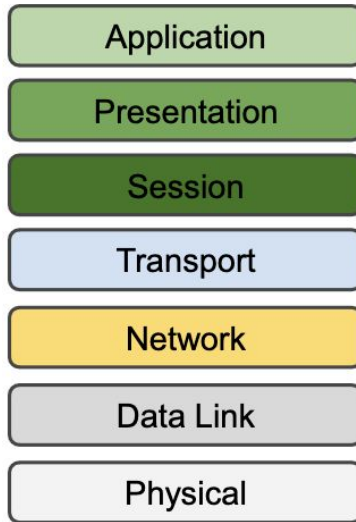
- Each layer is responsible for adding a header or a footer to the data being transferred
- The encapsulation process creates a Protocol Data Unit (PDU)

TCP/IP

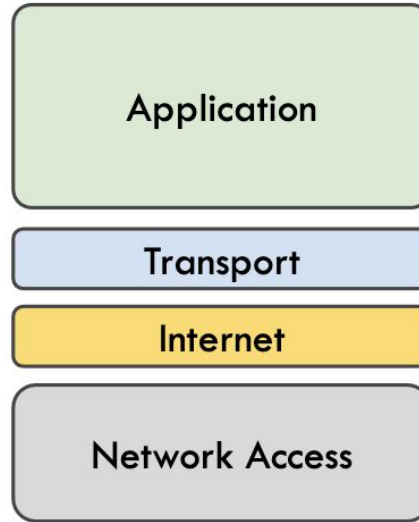
TCP/IP provides an alternative model used for the description of all network communications

- **Transmission Control Protocol (TCP)** which also implements the Transport layer of ISO/OSI model
- **Internet Protocol (IP)** which also implements the Network layer of ISO/OSI model

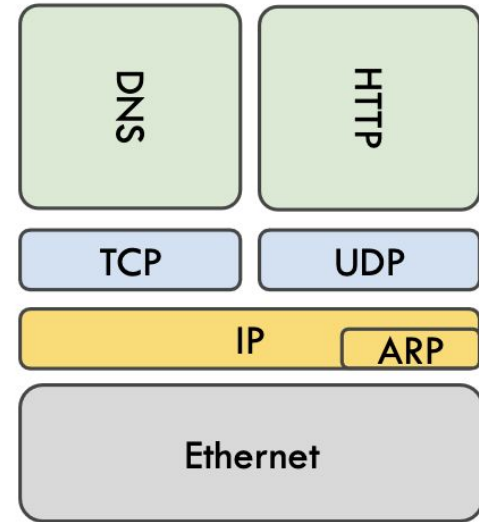
ISO/OSI and TCP/IP



ISO/OSI



TCP/IP



Standard protocols

Layer 2: Ethernet and bridges

Ethernet

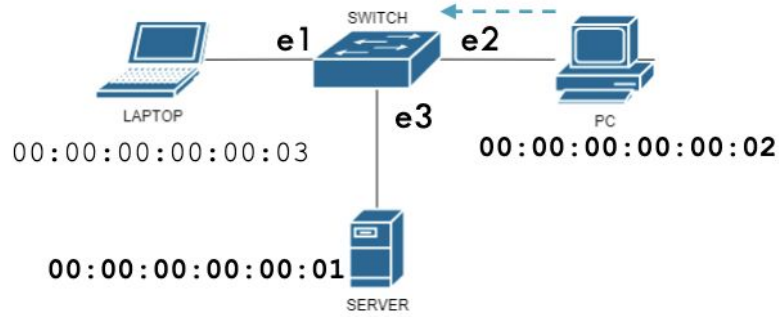
- Encapsulate data and transmit them in the form of frames
 - Frames leverage the Media Access Control (MAC) addresses
 - 48 bits burned in the adapter ROM (first 3-bytes: the ID of the manufacturer)
 - Every Ethernet device has a unique MAC address on its local network
 - A Frame includes the MAC address of the source and destination interface

Layer 2: Ethernet and bridges

Bridges and Switches

- analyze all frames received, find the destination MAC address, and forward them to the appropriate port
- To determine where to forward the traffic, they use a special table (MAC address table)

A basic switched network



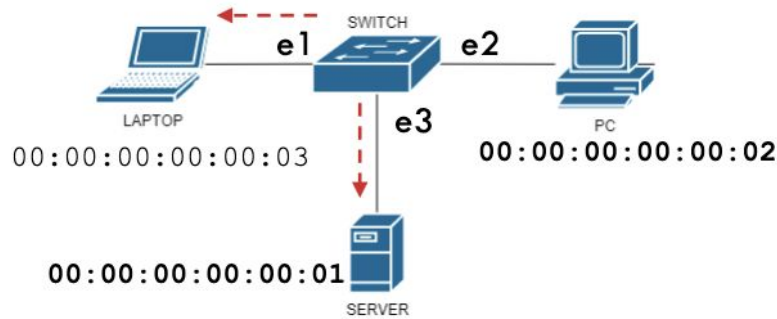
PC wants to send traffic to SERVER that has MAC address 00:00:00:00:00:01

- Creates a frame
- Sends it off toward the switch

MAC address table (switch)

MAC address	Port

A basic switched network



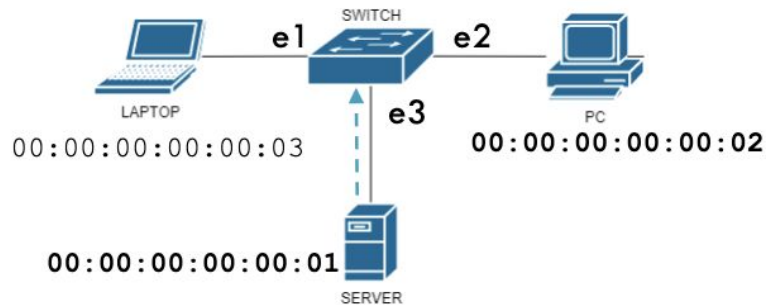
MAC address table (switch)

MAC address	Port
00:00:00:00:00:02	e2

The switch receives the traffic

- Creates a new entry
- Performs a lookup on its MAC address table to determine whether it knows which port to send the traffic to
- Since no matching entries exist, it broadcast the frame

A basic switched network

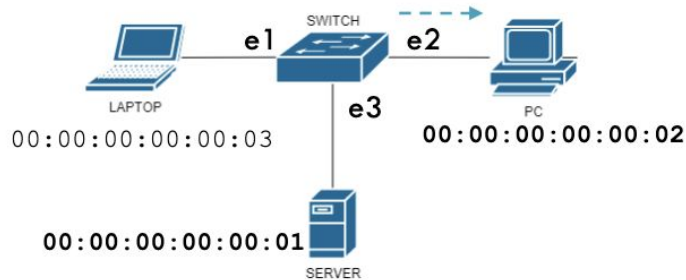


MAC address table (switch)

MAC address	Port
00:00:00:00:00:02	e2
00:00:00:00:00:01	e3

- The server sends a new frame back toward the switch containing 00:00:00:00:00:01 as the source address and 00:00:00:00:00:02 as the destination address
- The switch would receive the frame and create a new entry in its MAC address table for the Server MAC address

A basic switched network

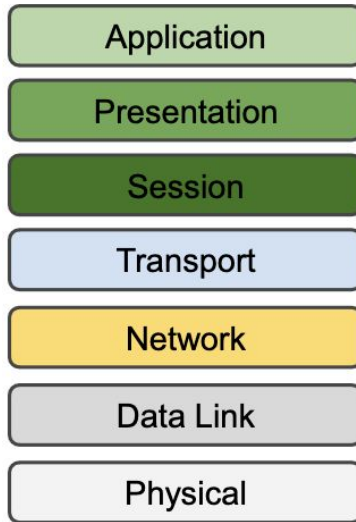


MAC address table (switch)

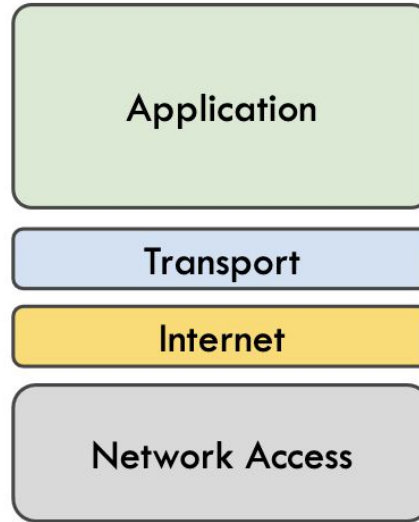
MAC address	Port
00:00:00:00:00:02	e2
00:00:00:00:00:01	e3

- Switch performs a lookup of its MAC address table to determine whether it knows which port to send the server frame to
- it sends the return traffic out only its e2 port (PC)

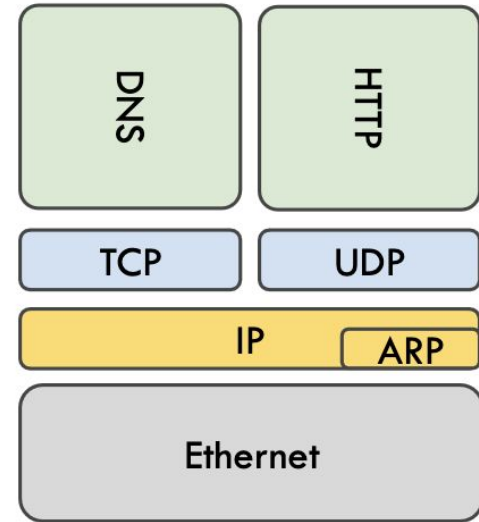
ISO/OSI and TCP/IP



ISO/OSI



TCP/IP



Standard protocols

Layer 3: IP and routing

Internet Protocol (IP)

- The standard for routing packets across interconnected networks
- Encapsulate data and pass that data in the form of packets

An Internet Protocol address is also known as an IP address

- A numerical label which assigned to each device connected to a computer network
- IPv4: 32 bits (grouped 8 bits at a time)

11000000 10101000 01100100 11001000

192 . 168 . 100 . 200

IP address and Netmask

An IP address has two components

- a network component
- a host component

The purpose of the netmask is to split the IP address into the two components

- when you combine, using a logical AND, the IP address and the netmask you reveal the network component

	11000000	10101000	01100100	11001000	address
	192	. 168	. 100	. 200	
	11111111	11111111	11111111	00000000	netmask (/24)
	255	. 255	. 255	. 0	
network	11000000	10101000	01100100	00000000	host
	192	. 168	. 100		

Reserved IP addresses

In every network, two addresses are used for special purposes

- Network address: is the first address in the network (all the host bits are 0) and it is used for identifying the network
- Broadcast address: is the last address in the network (all the host bits are 1)

Reserved IP addresses

Private IP addresses

- Range from 10.0.0.0 to 10.255.255.255 (a 10.0.0.0 network with a 255.0.0.0)
- Range from 172.16.0.0 to 172.31.255.255 (a 172.16.0.0 network with a 255.240.0.0)
- A 192.168.0.0 to 192.168.255.255 range (a 192.168.0.0 network masked by 255.255.0.0)

IP Routing

IP routing is the process of sending packets from a host on one network to another host on a different remote network

- Nodes use **routing tables** to determine the path to the destination network

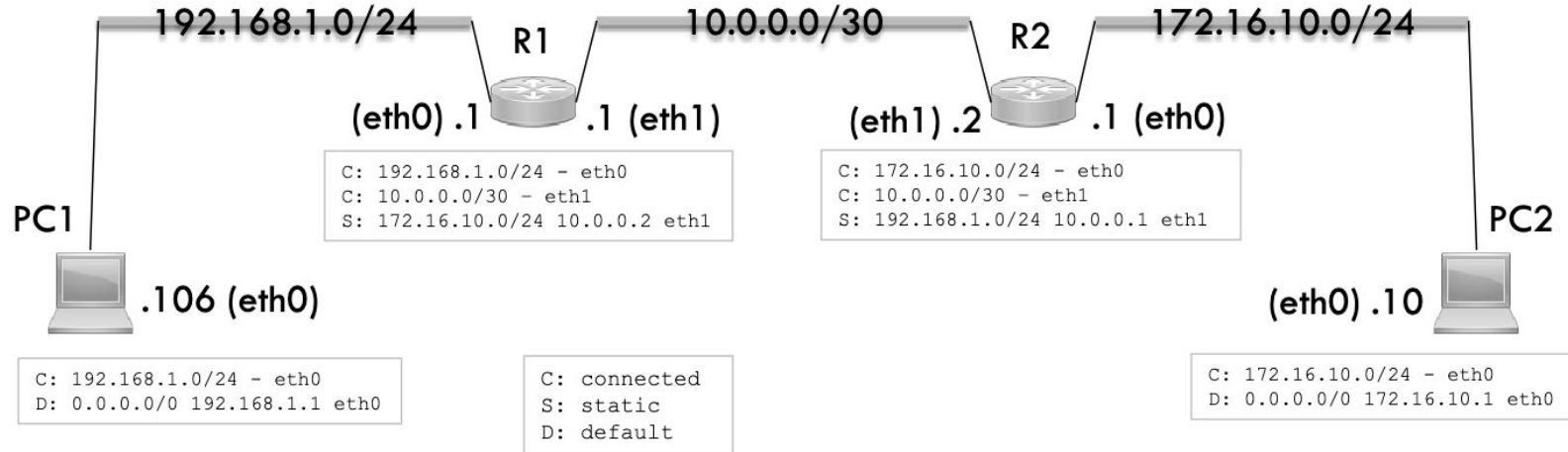
Note: If an IP node wants to communicate with a system in the same layer 2 domain, it checks its neighbor table, or ARP table

Routing Table

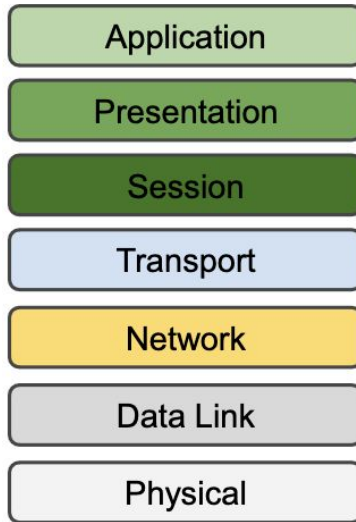
Each routing table consists of the following entries

- Network destination and subnet mask – specifies a range of IP addresses
- Remote router – IP address of the router used to reach that network
- Outgoing interface – outgoing interface the packet should go out to reach the destination network

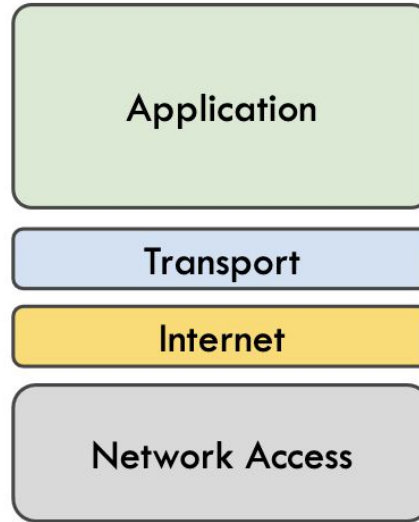
Routing tables



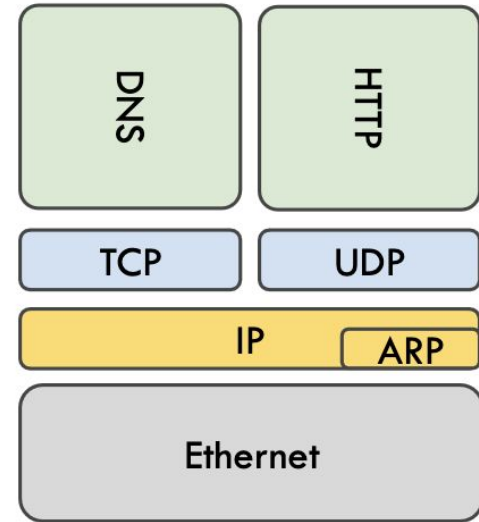
ISO/OSI and TCP/IP



ISO/OSI



TCP/IP

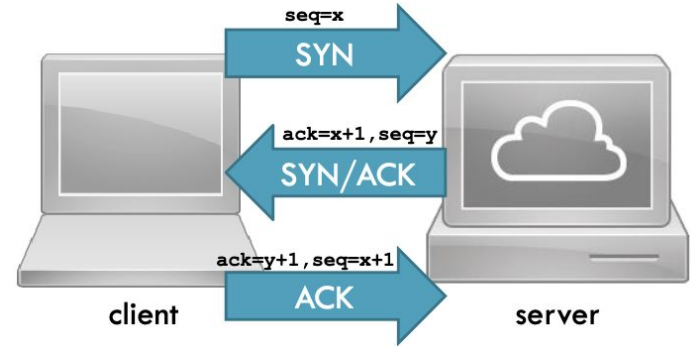


Standard protocols

TCP and UDP

TCP advantages

- creates a connection before any message is sent
- includes error recovery
- rearranges data packets in the specific order
- uses Three-Way Handshake (sequence (seq) and acknowledgment (ack) allows to detect missing segments)



UDP advantages

- simpler and faster than TCP (no connection state, small packet header, ...)
- used for streaming audio and video (can handle some packet loss)

Ports

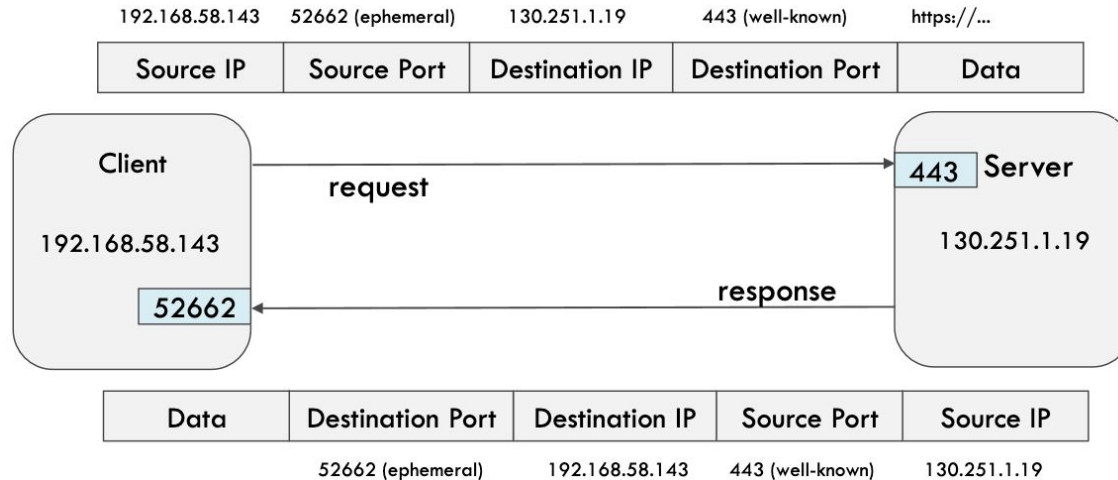
Transmitter and receiver are identified using ports (16-bit unsigned integer 0-65535)

- Well-known ports (0-1023): used by system processes
- Registered ports (1024-49151): assigned by a central authority
- Ephemeral ports (49152-65535): contain dynamic or private ports

Note: any service can listen on any port

Client - Server model

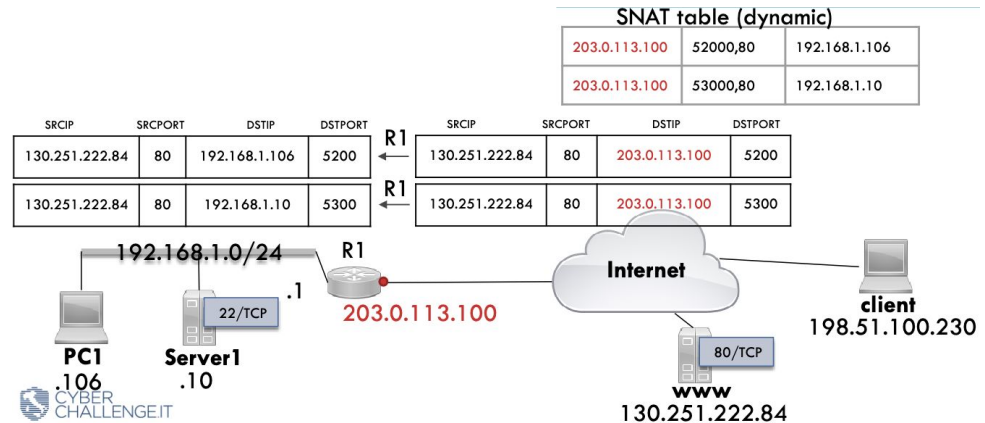
TCP/IP relies on the client-server model for enabling the process communication between network nodes



Network Address Translation

Rewriting the source and/or destination addresses of IP packets as they pass through a router or firewall

- private network are not routable
- translate a private address range to a single public IP address



Port forwarding

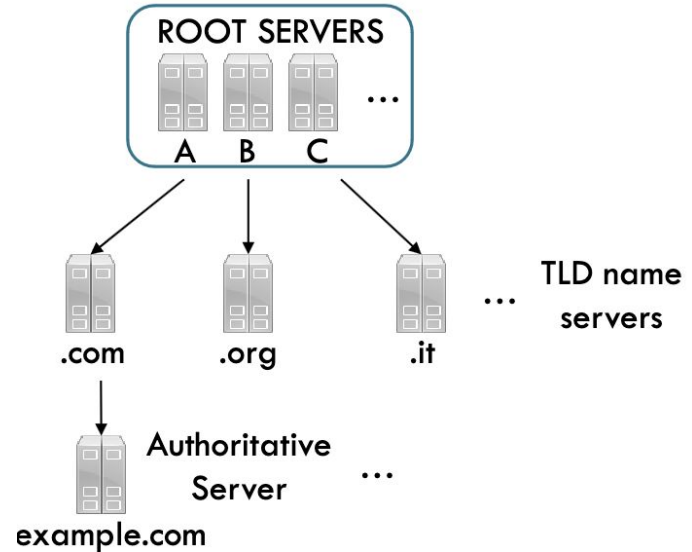
Port forwarding is a destination NAT rule, i.e., it is related to the destination address of a packet

- maps external IP addresses and ports to Internal IP addresses and ports, allowing access to internal services from the Internet

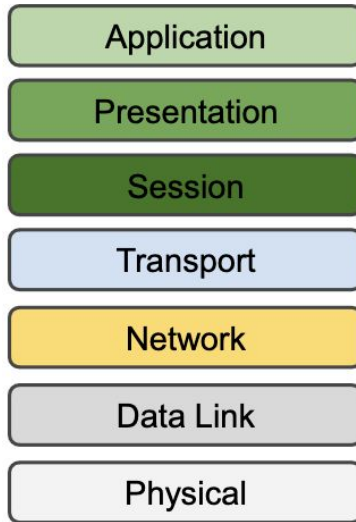
DNS - Domain Name System

DNS converts human readable domain names into IP addresses

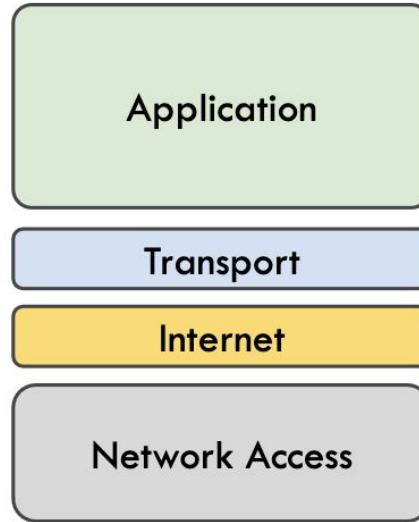
- An application or device (DNS client) issues a DNS address lookup (query), providing a hostname such as “example.com”



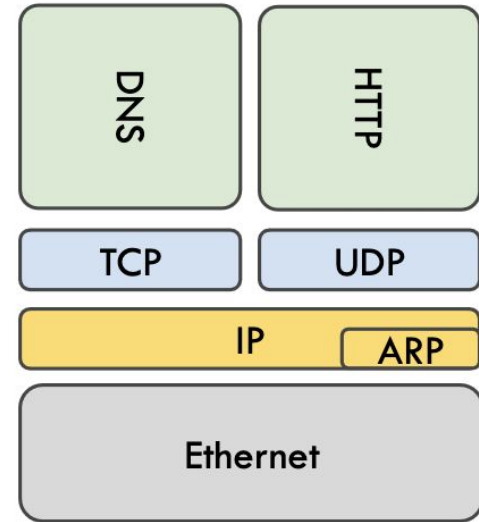
ISO/OSI and TCP/IP



ISO/OSI



TCP/IP



Standard protocols

HTTP - Hypertext Transfer Protocol

Client-server protocol which allows the fetching of resources, such as HTML documents

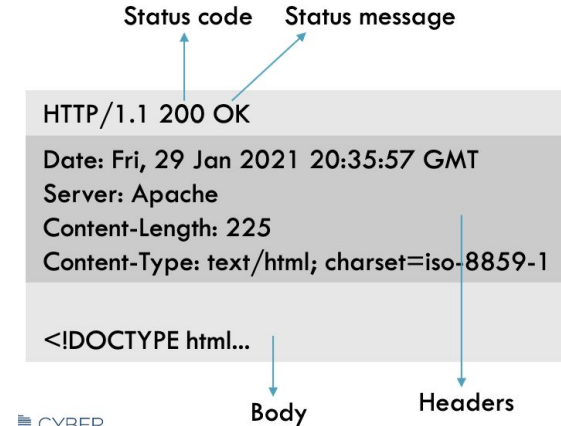
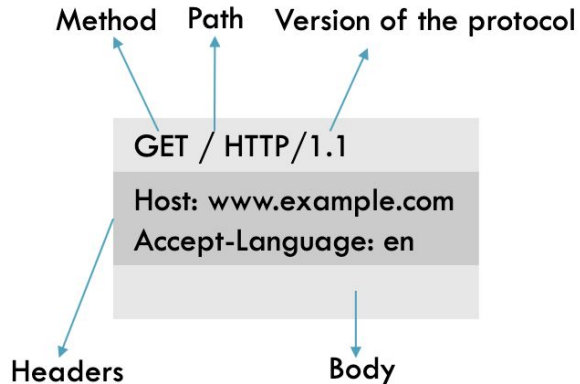
- Requests are sent by one entity, namely the user-agent
- the server provides the document requested by the client

<code>http://</code>	<code>www.example.com</code>	<code>:80</code>	<code>/path/to/myfile.html</code>	<code>?key1=value1&key2=value2</code>	<code>#SomewhereInTheDoc</code>
the protocol to be used	the name of the web server	the port (usually omitted if it is the well-known)	the path to the resource on the web server.	extra parameters provided to the web server	fragment identifier: refers to a specific location within the resource being returned.

HTTP - Hypertext Transfer Protocol

HTTP messages

- **HTTP Requests:** sent by the client to trigger an action on the server
- **HTTP Responses:** the answer from the server



Virtualization

Traditional Deployment

- Physical Hosts run an Operating System

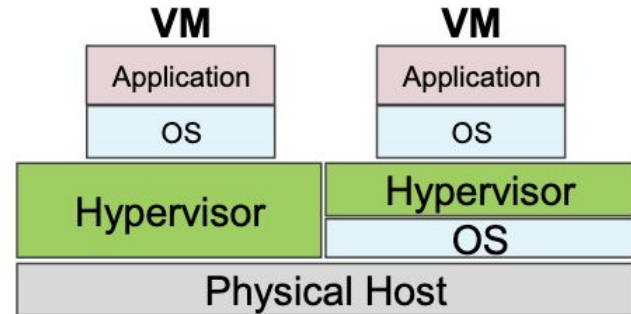


Traditional vs Virtualized Deployment

- Physical Hosts run an Operating System



- VM is a full machine running all the components, including its own Operating System, on top of the virtualized hardware

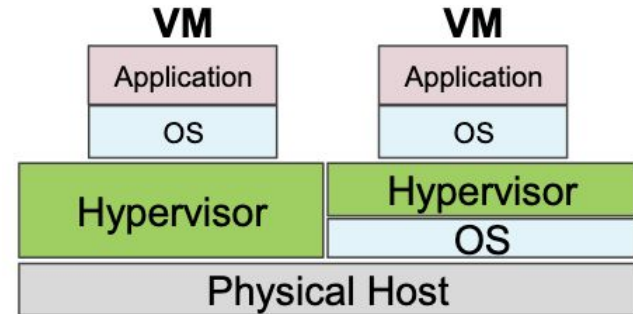


Traditional vs Virtualized Deployment

- No way to define resource boundaries for applications
- Isolating applications requires running them on different physical servers



- Applications are isolated between VMs
- Better utilization of the physical server resources
- Better scalability



Hardware Virtualization

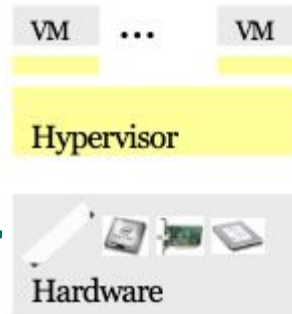
1st Generation: Full
virtualization (Binary
rewriting)

- Software Based
- VMware and Microsoft



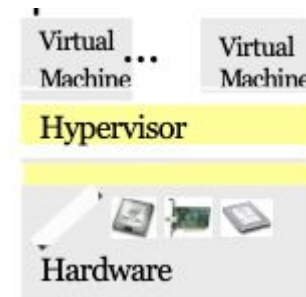
2nd Generation:
Paravirtualization

- Cooperative virtualization
- Modified guest
- VMware, Xen



3rd Generation:
Hardware-assisted
virtualization

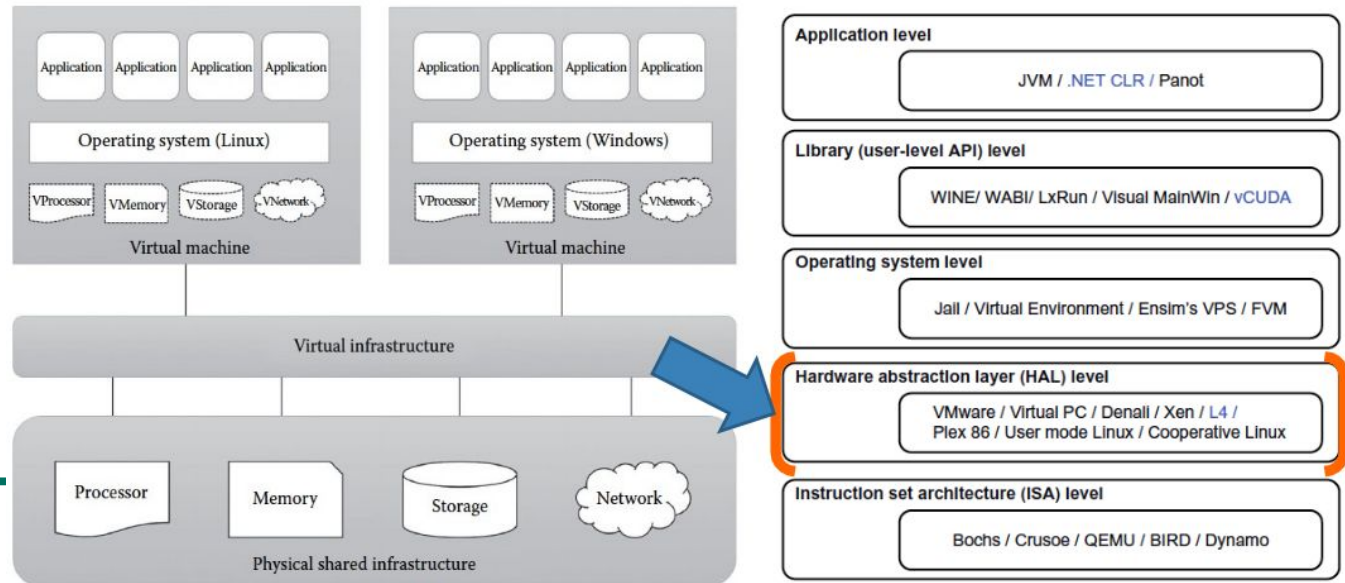
- Unmodified guest
- VMware and Xeon
- virtualization-aware hw platforms



Virtualization Layers

Virtualize the host machine's HW as virtual resources to be used by guest VMs

- virtualization layers can be interposed at various levels of a computer system

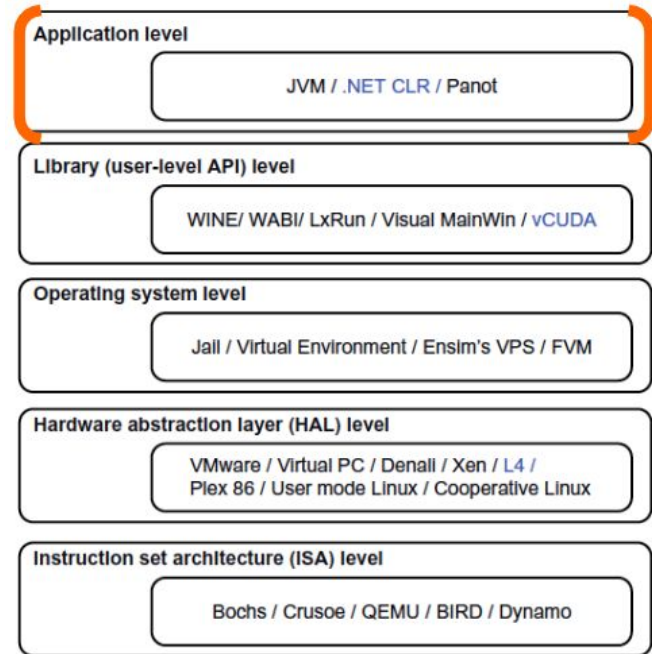


Virtualization Layers

User-Application (process) level
virtualization

- can run programs written and compiled to a particular abstract machine

Example: Java Virtual Machine (JVM)

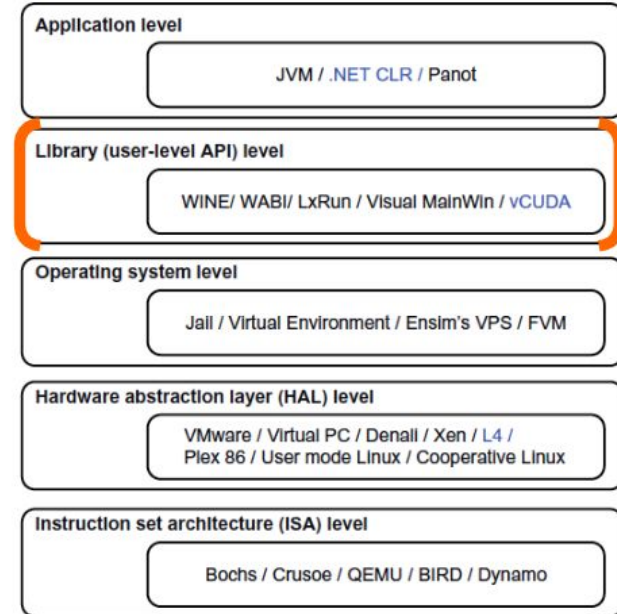


Virtualization Layers

Library-level virtualization

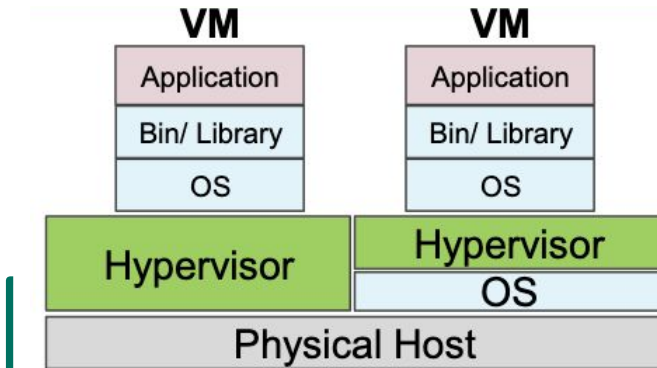
- running alien programs on a platform rather than creating a VM to run the entire operating system
- API call interception and remapping are the key functions performed

Example: Wine



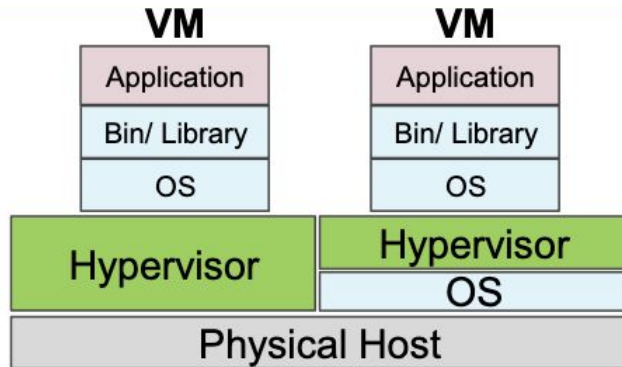
Virtualized Deployment

- Each VM has an OS and Application
- Share hardware resource from the Physical Host

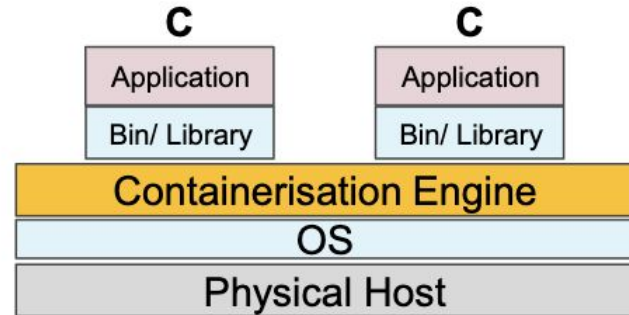


Virtualized vs Container Deployment

- Each VM has an OS and Application
- Share hardware resource from the Physical Host

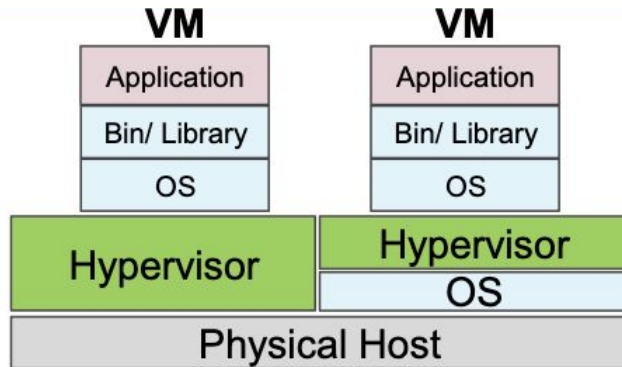


- Isolated environments sharing the same real operating system
- Distinct image that provides all files necessary to support them

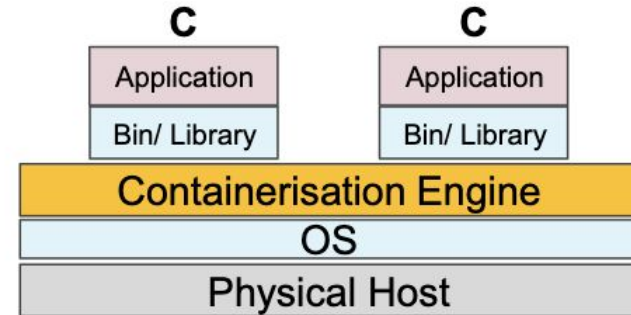


Virtualized vs Container Deployment

- Each VM relies on a full copy of an Operating System
- Best when there are applications that need to run on different OS



- Lightweight: sharing OS resources
- Provides a process-level isolation

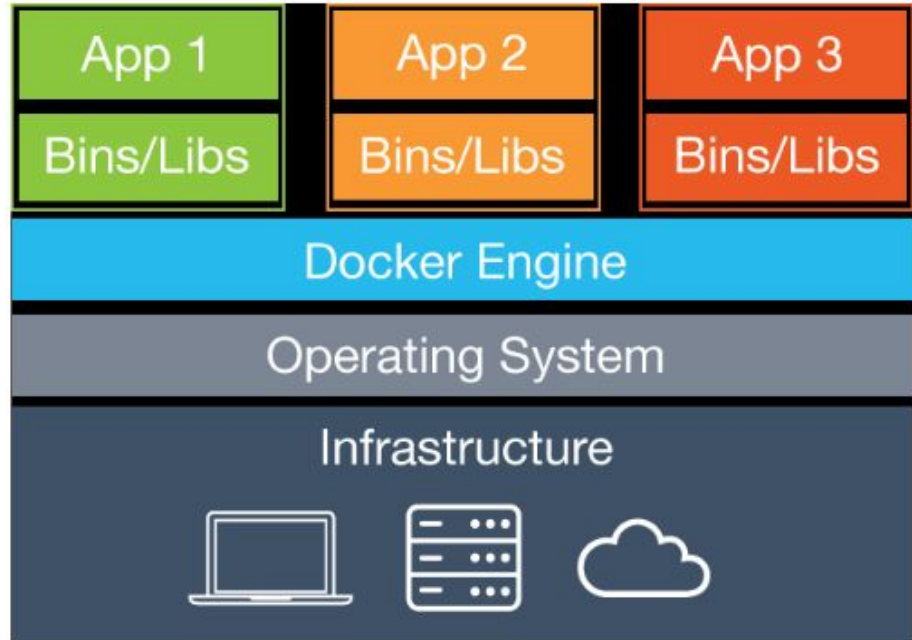


Docker

Docker

Lightweight, open and secure
container-based virtualization

- containers include the application and all of its dependencies
- containers run as an isolated process
- containers are also not tied to any specific infrastructure



Docker



Tools that enable rapid development of containers

- an approach to combine pre built code modules into a single image that runs when a container is deployed
- all sw needed for a container, including the app to run, is placed in an image file
- when an image runs, a container is created to run the app, and the app is said containerized

Docker Hub

Extensive registry of SW for use with containers

- Docker Hub
- users can combine pieces from the registry as they were modules of a library

Docker Hub [Give feedback](#)

GenAI catalog

[View catalog](#)**ai/meta-llama**

Meta's Llama with vllm runner for CUDA-accelerated, advanced NLP applications.

↓ 3.5K ☆ 17

**ai/mistral**

Mistral's vllm-powered, high-speed conversational AI for real-time CUDA performance.

↓ 2.2K ☆ 3

**ai/phi**

Specialized Phi 3 NLP with vllm runner, ideal for targeted language tasks on CUDA.

↓ 363 ☆ 3

Machine Learning & AI

[View category](#)**ollama/ollama**

The easiest way to get up and running with large language models.

↓ 10M+ ☆ 1.2K

**pytorch/pytorch**

PyTorch is a deep learning framework that puts Python first.

↓ 10M+ ☆ 1.4K

**langchain/langchain****tensorflow/tensorflow**

Docker

Techniques that allow rapid instantiation of an app

- a container is smaller than a VM
- no need to wait for an OS to boot, containers start instantly

Reproducible execution across hosts

- containers are immutable, independent of the number of time the image runs is a container
- containers perform the same on any system

Docker Engine

A client-server application

- Server
 - containers operate under the control of an application known as docker daemon
- Client
 - a user interface app referred to as docker, that provides a command line interface

Docker

Terminology

- Image
 - a file that contains binary code for a container along with its dependencies
- Layer
 - One piece of sw added to an image as it is built
- Container
 - An instance of an image
- Dockerfile
 - a specification for how to build an image

Docker images and containers

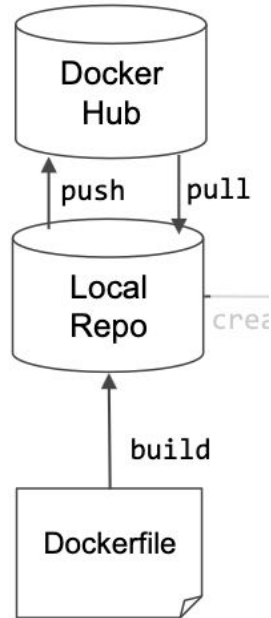
An image is an Immutable template for containers

- it includes everything needed to run an application (code, runtime, system tools, system libraries, and settings)

A container is an instance of an image

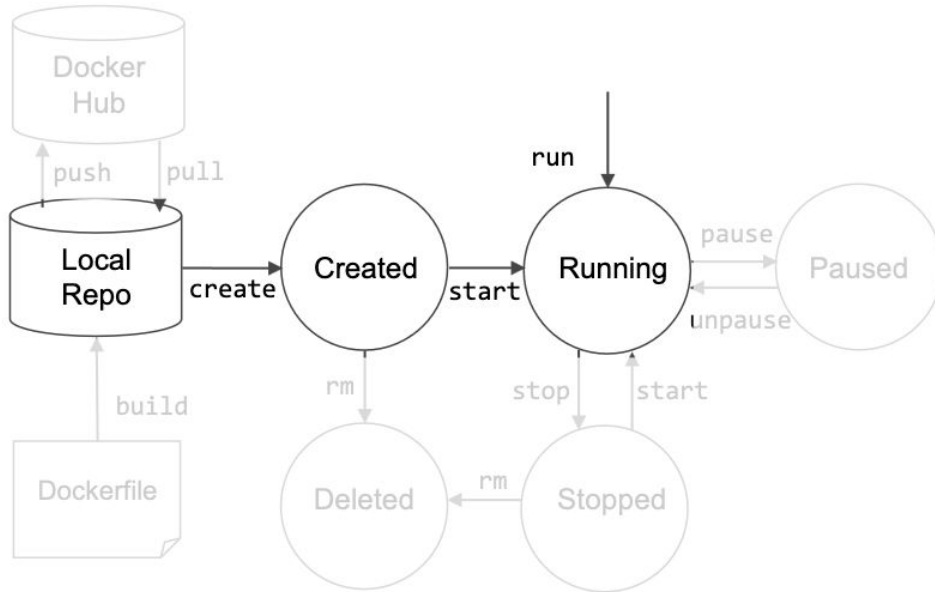
- all changes made to the running container (e.g., writing new files or modifying existing files) are written to a new new writable layer

Docker container lifecycle



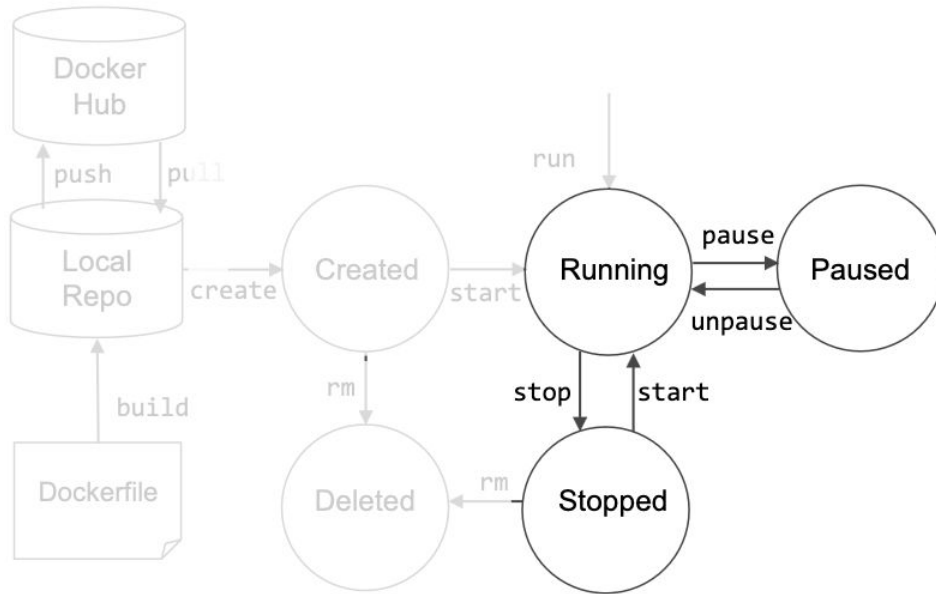
- Docker images can be pulled from a central repository
- Pulled images are saved in a local repository
- Custom images can be created and saved starting from a specific configuration file (Dockerfile)
- Custom images can be pushed to the central repository

Docker container lifecycle



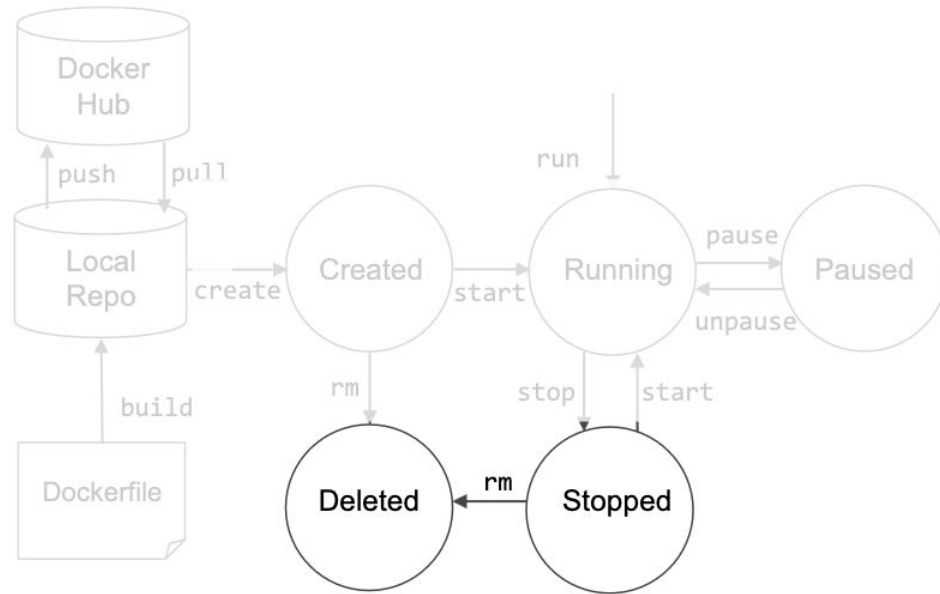
- A saved image can be used for creating a container
- Starting a container means running a default command contained in the image (the entrypoint)

Docker container lifecycle



- After executing the entrypoint, the container stops
- A foreground process specified as the entrypoint can keep running the container
- A running container can be paused or stopped

Docker container lifecycle



- After a container is stopped, the writeable layer still exists
- Deleting a container permanently removes the associated writable layer

Docker Commands

Pull an image from the central hub

- `docker pull <image>`

Build an image from a Dockerfile

- `docker build -t <image>`

List images saved in the local repository

- `docker images`

Delete an image

- `docker rmi <image>`

Docker Commands

Start a container

- `docker run <image>`

Execute a command `<cmd>` (e.g., `/bin/bash`) in a running container

- `docker exec -it <containerID> <cmd>`

Stop a container

- `docker stop <containerID>`

Remove a container

- `docker rm <containerID>`

Docker Commands

List running containers

- `docker ps`

List running and stopped containers

- `docker ps -a`

Logs

- `docker logs <container>`

Docker Cheatsheet

 Cheatsheet for Docker CLI			
Run a new Container	Manage Containers	Manage Images	Info & Stats
Start a new Container from an Image <pre>docker run IMAGE docker run nginx</pre> ...and assign it a name <pre>docker run --name CONTAINER IMAGE docker run --name web nginx</pre> ...and map a port <pre>docker run -p HOSTPORT:CONTAINERPORT IMAGE docker run -p 8080:80 nginx</pre> ...and map all ports <pre>docker run -P IMAGE docker run -P nginx</pre> ...and start container in background <pre>docker run -d IMAGE docker run -d nginx</pre> ...and assign it a hostname <pre>docker run --hostname HOSTNAME IMAGE docker run --hostname srv nginx</pre> ...and add a dns entry <pre>docker run --add-host HOSTNAME:IP IMAGE</pre> ...and map a local directory into the container <pre>docker run -v HOSTDIR:TARGETDIR IMAGE docker run -v ~/.usr/share/nginx/html nginx</pre> ...but change the entrypoint <pre>docker run -it --entrypoint EXECUTABLE IMAGE docker run -it --entrypoint bash nginx</pre>	Show a list of running containers <pre>docker ps</pre> Show a list of all containers <pre>docker ps -a</pre> Delete a container <pre>docker rm CONTAINER docker rm web</pre> Delete a running container <pre>docker rm -f CONTAINER docker rm -f web</pre> Delete stopped containers <pre>docker container prune</pre> Stop a running container <pre>docker stop CONTAINER docker stop web</pre> Start a stopped container <pre>docker start CONTAINER docker start web</pre> Copy a file from a container to the host <pre>docker cp CONTAINER:SOURCE TARGET docker cp web:/index.html index.html</pre> Copy a file from the host to a container <pre>docker cp TARGET CONTAINER:SOURCE docker cp index.html web:/index.html</pre> Start a shell inside a running container <pre>docker exec -it CONTAINER EXECUTABLE docker exec -it web bash</pre> Rename a container <pre>docker rename OLD_NAME NEW_NAME docker rename 096 web</pre> Create an image out of container <pre>docker commit CONTAINER docker commit web</pre>	Download an image <pre>docker pull IMAGE[:TAG] docker pull nginx</pre> Upload an image to a repository <pre>docker push IMAGE docker push myimage:1.0</pre> Delete an image <pre>docker rmi IMAGE</pre> Show a list of all Images <pre>docker images</pre> Delete dangling images <pre>docker image prune</pre> Delete all unused images <pre>docker image prune -a</pre> Build an image from a Dockerfile <pre>docker build DIRECTORY docker build .</pre> Tag an image <pre>docker tag IMAGE NEWIMAGE docker tag ubuntu ubuntu:18.04</pre> Build and tag an image from a Dockerfile <pre>docker build -t IMAGE DIRECTORY docker build -t myimage .</pre> Save an image to .tar file <pre>docker save IMAGE > FILE docker save nginx > nginx.tar</pre> Load an image from a .tar file <pre>docker load -i TARFILE docker load -i nginx.tar</pre>	Show the logs of a container <pre>docker logs CONTAINER docker logs web</pre> Show stats of running containers <pre>docker stats</pre> Show processes of container <pre>docker top CONTAINER docker top web</pre> Show installed docker version <pre>docker version</pre> Get detailed info about an object <pre>docker inspect NAME docker inspect nginx</pre> Show all modified files in container <pre>docker diff CONTAINER docker diff web</pre> Show mapped ports of a container <pre>docker port CONTAINER docker port web</pre>

Docker volumes

Volumes can be used to save (persist) data and to share data between containers. Volume is unrelated to the container layers: deleting a container does not involve deleting an associated volume

Create a named volume

- `docker volume create volumename`

Run a container using the named volume

- `docker run -v volumename:/path/in/container_filesystem`

Running a container using a host volume

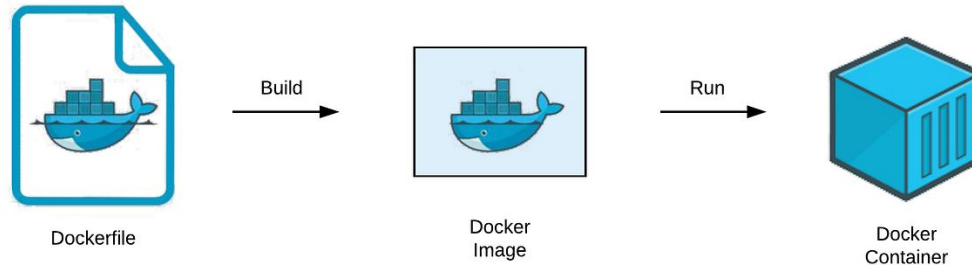
- `docker run -v /path/on/host_filesystem:/path/in/container_filesystem`

Dockerfile

Docker can build custom images automatically by reading the instructions from a Dockerfile

Dockerfile

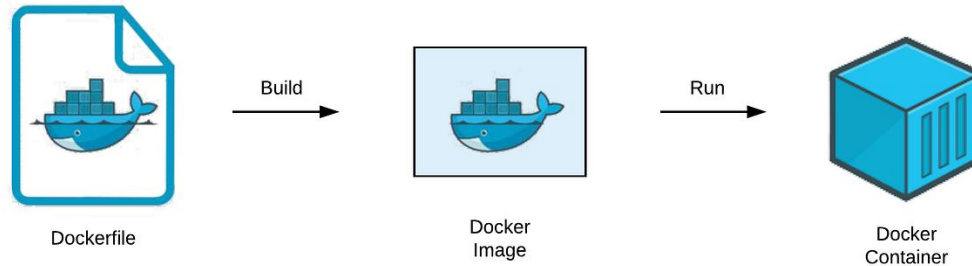
- text document that contains all the commands a user could call on the command line to assemble an image



Dockerfile

List of commands

- each instruction adds a new layer to the image and then commits the image



Dockerfile

- FROM <image>
 - specify a base operating system for the image
- RUN <cmd>
 - run the specified command
- COPY <src> <dest>
 - copy a file to the image file system
- CMD <"exec", "param1", "param2">
 - configure the default command when container starts

```
FROM ubuntu
```

```
RUN apt install apache2
```

```
COPY vh.conf /etc/apache2/conf/
```

```
CMD ["apache2", "-D", "  
FOREGROUND "]
```

Dockerfile

- ENTRYPOINT
 - specify an app that runs when the container starts
- EXPOSE
 - specifies the network ports used at runtime by the container
- VOLUME
 - specifies mount points for an external file system

Dockerfile

```
FROM openjdk:8
```

```
USER root
```

```
EXPOSE 22/tcp
```

```
EXPOSE 22/udp
```

```
EXPOSE 8281
```

```
RUN apt-get update && \  
apt-get install -y --no-install-recommends nano
```

```
# ----- Config Manager -----
```

```
USER root
```

```
RUN useradd -m -s /bin/bash pubmed
```

```
RUN mkdir -p /home/pubmed/config
```

```
COPY config/*.properties /home/pubmed/config/
```

```
RUN chown pubmed /home/pubmed/config
```

```
WORKDIR /home/pubmed
```

```
USER pubmed
```

```
RUN echo 'Downloading Spring Worker..'
```

```
COPY config/PubMedKnowledgeGraph_Manager-v1.0.jar PubMedKnowledgeGraph_Manager-v1.0.jar
```


Dockerfile

```
# ----- Config Manager -----  
USER root  
COPY config/entrypoint.sh /usr/local/bin/  
RUN chmod +x /usr/local/bin/entrypoint.sh && ln -s /usr/local/bin/entrypoint.sh /  
ENTRYPOINT ["entrypoint.sh"]
```

```
#!/bin/bash
```

```
cd /home/pubmed/  
java -jar PubMedKnowledgeGraph_Manager-v1.0.jar
```

Dockerfile

```
1 FROM ubuntu:22.04
2
3 RUN apt-get update && apt-get install -y \
4     ca-certificates \
5     curl
6
7 WORKDIR /test_app
8
9 RUN npm install --legacy-peer-deps
10
11 ARG HOST
12 ENV HOST $HOST
13
14 RUN npm run build
15
16 COPY entrypoint.sh /usr/local/bin/entrypoint.sh
17 RUN chmod +x /usr/local/bin/entrypoint.sh
18
19 CMD ["sh", "/usr/local/bin/entrypoint.sh"]
```

Docker networking

Docker supports different configurations

- Bridge
 - isolated layer 3 networks enabling connected containers to communicate
 - not expose any of its port, but can be published and mapped to a port on the Docker host
- Host
 - containers use the host network
 - listening ports are exposed to the outside world

Docker networking

Create network with a configured subnet and gateway address

- `docker network create --driver bridge <networkname> --subnet=<ip/mask> --gateway=<ip/mask>`

Connect a container to a network

- `docker network connect <networkname> <containerid>`

Run a container and expose a port

- `docker run -h <host-name> -p <internal-port>:<exposed-port> --name <container-name> <image-name>`

Docker Compose

Docker Compose is a tool for defining and running multi-container Docker applications

- A single file for providing configurations
- A single set of commands for configuring, building, and running all the containers
- human-readable syntax (YAML)



Docker Compose

Defines

- Services: configuration that is applied to each container
- Networks (optional): define configuration of networks to be created
- Volumes (optional): define configuration of volumes to be created



Docker Compose

run services

- `docker-compose up`

stop services

- `docker-compose stop`

start services

- `docker-compose start`

stop and remove containers and networks

- `docker-compose down`

show logs

- `docker-compose logs -f
[service_name]`

Docker and Docker-Compose Cheatsheet

CheatSheet for Docker

Docker base controls

List of containers started

`docker ps`

List of all containers

`docker ps -a`

Recovers the docker configuration

`docker info`

Recover docker version

`docker version`

Builds a Docker image

from a "Dockerfile" or a container.

`docker build -t < Image>: <Tag>`

Connects to a remote repository

`docker login < repository>`

Push image to remote repository

`docker push < Image_Name>: <Tag>`

Extracts the image from

the remote repository

`docker pull < Image_Name>`

Network commands

List of networks

`docker network ls`

Control network information

`docker network inspect < Network>`

Create a network

`docker network create < Network>`

Removes a network

`docker network rm < Network>`

Connect a container to the network

`docker network connect`

`< Network> < Container>`

Specifies the IP address of the

container interface

`docker network connect --ip`

`< IP> < Network> < Container>`

Disconnect the network container

`docker network disconnect`

`< Network_Name> < Container>`

Image Control

List locally available images

`docker images`

Execute the image

`docker run < Image>`

Create an image

`docker create < Image>: <Tag>`

Delete image

`docker rmi < Image>`

Saving images in a tar archive

`docker save < Image>`

Search for Docker images

`docker search < image>`

Recover a docker image

`docker pull < image>`

Builds an image from a "dockerfile"

`docker build -t <image-name>`

Removes all unnecessary docking images

`docker image prune`

Volume controls

Image controls

`docker volume ls`

Volume control

`docker volume inspect < Volume>`

Create a volume

`docker volume create < Volume>`

Remove volume

`docker volume rm < Volume>`

Removes unused volumes

`docker volume prune`

Docker-Compose

Start all services defined in the `docker-compose.yml` file.

`docker-compose up`

Stop and remove all containers, networks, and volumes associated with the services defined in the `docker-compose.yml` file.

`docker-compose down`

List all containers started by the `docker-compose up` command, along with their status.

`docker-compose ps`

View the logs of all containers or a specific service.

`docker-compose logs`

Build or rebuild all Docker images defined in the `docker-compose.yml` file.

`docker-compose build`

Stop all containers started by `docker-compose up` command without removing them.

`docker-compose stop`

Restart all containers or a specific service.

`docker-compose restart`

Run a command in a running container.

`docker-compose exec`

Pull updated images for services from their respective repositories.

`docker-compose pull`

Scale the number of containers for a specific service.

`docker-compose up --scale`

`<service>=<n>`

Docker Compose

```
1 services:
2   test_api:
3     image: europe-west8-docker.pkg.dev/test_api:latest
4     pull_policy: always
5     depends_on:
6       db:
7         condition: service_healthy
8     redis:
9       condition: service_healthy
10    environment:
11      DATABASE_PASSWORD: ${DATABASE_PASSWORD}
12      DATABASE_URL: postgres://postgres:${DATABASE_PASSWORD}@db:5432/test_db
13      REDIS_URL: redis://redis:6379
14      LETSENCRYPT_HOST: domain.com
15    ports:
16      - "8080:8080"
17    networks:
18      - test-net
```

```
1 db:
2   image: timescale/timescaledb:latest-pg15
3   environment:
4     PGUSER: postgres
5     POSTGRES_USER: postgres
6     POSTGRES_PASSWORD: ${DATABASE_PASSWORD}
7     POSTGRES_DB: test_db
8   ports:
9     - "5432:5432"
10  volumes:
11    - pgdata:/var/lib/postgresql/data
12  networks:
13    - test-net
14  healthcheck:
15    test: ["CMD-SHELL", "pg_isready"]
16    interval: 10s
17    timeout: 5s
18    retries: 5
```

Docker Compose

```
1 nginx-proxy:
2   image: nginxproxy/nginx-proxy
3   container_name: nginx-proxy
4   ports:
5     - "80:80"
6     - "443:443"
7   volumes:
8     - vhost:/etc/nginx/vhost.d
9     - conf:/etc/nginx/conf.d
10    - html:/usr/share/nginx/html
11    - certs:/etc/nginx/certs:ro
12    - /var/run/docker.sock:/tmp/docker.sock:ro
13   networks:
14     - test-net
```

```
1 acme-companion:
2   image: nginxproxy/acme-companion
3   container_name: nginx-proxy-acme
4   environment:
5     - DEFAULT_EMAIL=test@domain.com
6   volumes_from:
7     - nginx-proxy
8   volumes:
9     - certs:/etc/nginx/certs:rw
10    - acme:/etc/acme.sh
11    - /var/run/docker.sock:/var/run/docker.sock:ro
12   networks:
13     - test-net
```

Docker Compose

```
1 networks:
2   test-net:
3     driver: bridge
4
5 volumes:
6   pgdata:
7     driver: local
8     driver_opts:
9       type: "none"
10      o: "bind"
11      device: "/mnt/data/pgdata"
12 vhost:
13 conf:
14 html:
15 certs:
16 acme:
17
```

Challenges

NS_0.00 - Level0 - Setup

NS_0.01 - Level1 - Nodes console

NS_0.02 - Level2 - A well-known port

NS_0.03 - Level3 - One well-known and one non-standard port

NS_0.04 - Level4 - 3-way handshake

NS_0.05 - Level5 - A route to the flag

NS_0.06 - Level6 - NAT1

NS_0.07 - Level7 - NAT2

NS_0.08 - Level8 - The final shot

NS_0.09 - Docker networks